

Software Engineering (2UCC501)

- Module 1: The Product and the Process
- **CO 1: Understand the software development process and Estimate different types of resources for the given project**
- 1.1 Software Development Life Cycle models: Waterfall, RAD, Spiral, Agile process
- 1.2 Understanding software process, Process metric, CMM levels
- 1.3 Planning & Estimation: Product metrics estimations- LOC, FP, COCOMO models
- 1.4 Project Management Activities: Planning, Scheduling & Tracking



Contents

- Introduction to software engineering
- Need of Software engineering
- Nature of Software
- Types of Software
- Software Process

Software & Program

- Program:
- A step by step instructions that accomplishes a specific task by using programming language i. e **a set of instructions.**
- Software:
- It is a program with proper documentation and user manuals which mainly consists installation guide and other manual (operating procedures).
- **Software= program + documentation**

Introduction to Software Engineering

- What is Software

Software is:

(1) instructions (computer programs) that when executed provide desired features, function, and performance;

(2) data structures that enable the programs to adequately manipulate information and

(3) documentation that describes the operation and use of the programs.

Software includes:

- Program/ Product :
 - Operating System
 - API/ Library Functions/ Apps
 - Databases
- Documents:
 - Problem Definition
 - Method to solve / approach
 - Code (Reference manual)
 - Users Manual
 - Test reports

Why Software is Important?

- The economies of ALL developed nations are dependent on software.
- More and more systems are software controlled (transportation, medical, telecommunications, military, industrial, entertainment,)
- Software engineering is concerned with theories, methods and tools for professional software development.
- Expenditure on software represents a significant fraction of GNP in all developed countries.

Software products

- **Generic products**

- Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- Examples – PC software such as editing, graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.

- **Customized products**

- Software that is commissioned by a specific customer to meet their own needs.
- Examples – embedded control systems, air traffic control software, traffic monitoring systems

Software—New Categories

- **Open world computing**—**pervasive**, distributed computing due to wireless networking. How to allow mobile devices, personal computer, enterprise system to communicate across vast network.
- **Netsourcing**—the Web as a computing engine. How to architect simple and sophisticated applications to target end-users worldwide.
- **Open source**—“free” source code open to the computing community (a blessing, but also a potential curse!)

Software Crisis

- Problems with software?
 - The major problem with current scenario in Software industry is, software usually overrun their development cost, they exceed their development ~~duration~~ limits, are usually of poor quality.
 - As long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.— Edsger Dijkstra, The Humble Programmer (EWD340), Communications of the ACM

What is Software Engineering

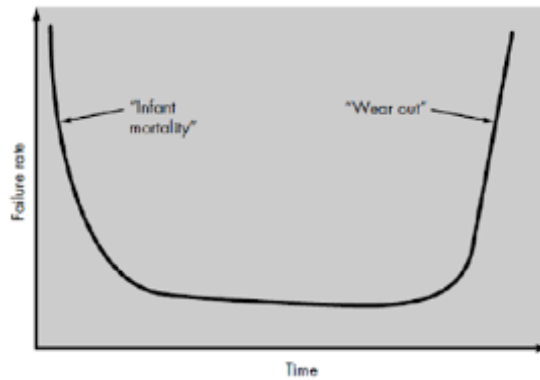
- In software industry we understand that there is an urgent need to use proper strategies, process and development cycles so that we can produce or we can develop quality products that are within budget, with in time and must satisfy the requirements of user.
- **Software Engineering** is the process of designing, developing, testing, and maintaining software.
- It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

Definitions of SE-

- IEEE defines software engineering as: (1) The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software; that is, the application of engineering to software. (2) Study of approaches as in (1).
- Fritz Bauer defined Software engineering as the “establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.”

Features/characteristics of Software?

- Software is developed or engineered:
 - Unlike physical products, it is not produce in factories through manufacturing process.
 - It is crafted through a process of coding and development.
- Software doesn't "wear out." (but it can deteriorates due to change):
 - Hardware has bathtub curve of failure rate (high failure rate in the beginning, then drop to steady state, then cumulative effects of dust, vibration, abuse occurs).
 - software's performance doesn't degrade due to physical use.



- Custom-Built Nature
- Intangibility of software

Nature of Software

- Software takes on a dual role.
- It is a product, and at the same time, the vehicle for delivering a product.
- As a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware
- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).

Software Applications Domains

- System software
- Application software
- Engineering/scientific software
- Embedded software
- Product-line software
- WebApps (Web applications)
- AI software



How the customer explained it



How the project leader understood it



How the engineer designed it



How the programmer wrote it



How the sales executive described it



How the project was documented



What operations installed



How the customer was billed



How the helpdesk supported it



What the customer really needed

Software Engineering Layers



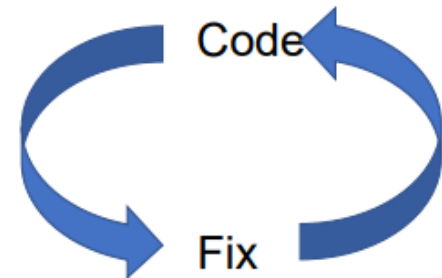
4 P's Related to Software Engineering (SE)

- Product
 - The deliverables
- Process
 - Development
 - Testing
- Project
 - Systematic approach with specific characteristics
- People
 - Designers, Analysts, Developers, Users

Generic Software Development Process Models

- Code & Fix Model
- Waterfall model
- Rapid Application Development (RAD)
- Spiral
- Iterative
- Evolutionary
- Agile

Code & Fix Model



- Code the software & Fix Modify (similar to trail & error)
- Generally used for SMALL projects
- Very short delivery time
- SE is restricted only to management of program
- NO way to keep track of requirements, business logic and design

Waterfall model

- The waterfall model is a continuous software development model in which development is seen as flowing steadily downwards (like a waterfall) through the steps of requirements analysis, design, implementation, testing (validation), integration, and maintenance.
- This model considers that one phase can be started after the completion of the previous phase.
- That is the output of one phase will be the input to the next phase.

Feasibility Study

Req. Analysis

Design

Coding

Testing

Maintenance

Classical Waterfall Model

Simplest and most intuitive

Advantages

- Requirements are clear.
- There are no ambiguous requirements
- It is good to use this model when the technology is well understood.
- The Project is short and cost is low. \
- Risk is zero and minimum

Disadvantages

- No working software is produced until late during the life cycle.
- High amount of risk and uncertainty.
- Not good model for complex and object oriented project.
- Poor model for long and ongoing projects.
- It is difficult to measure progress within stages

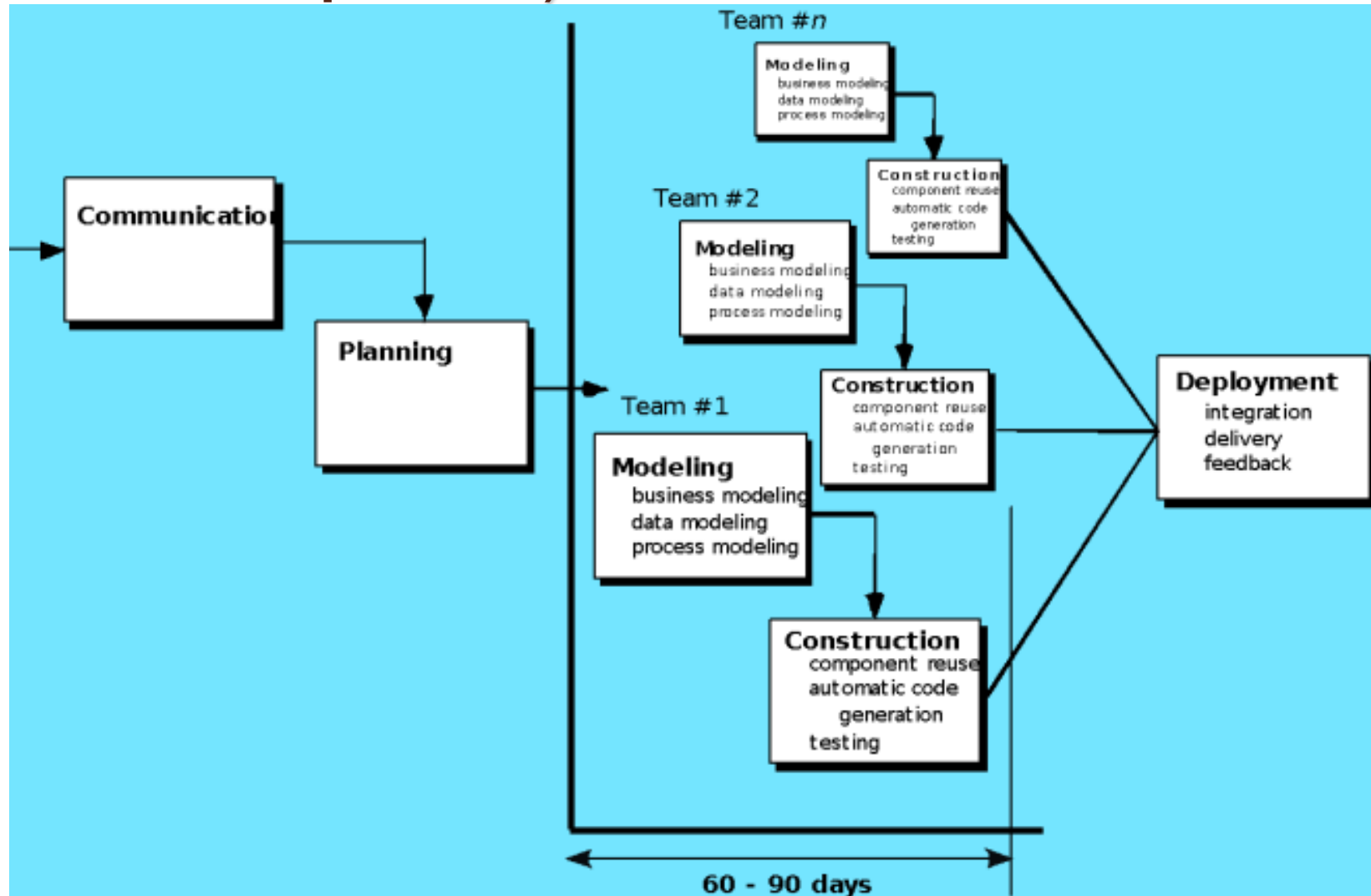
Rapid Application Development (RAD) Model

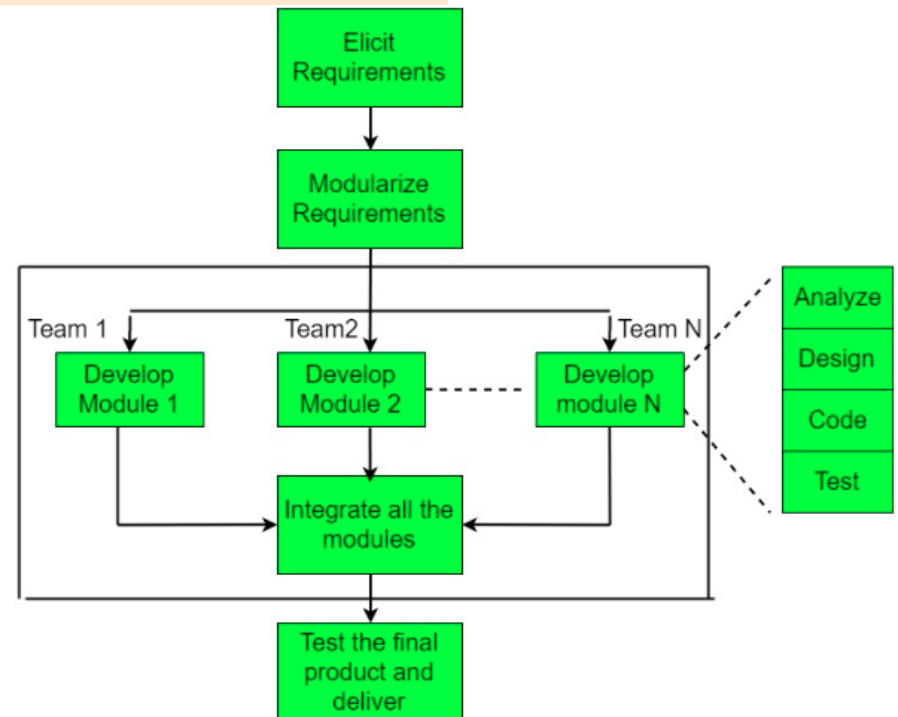
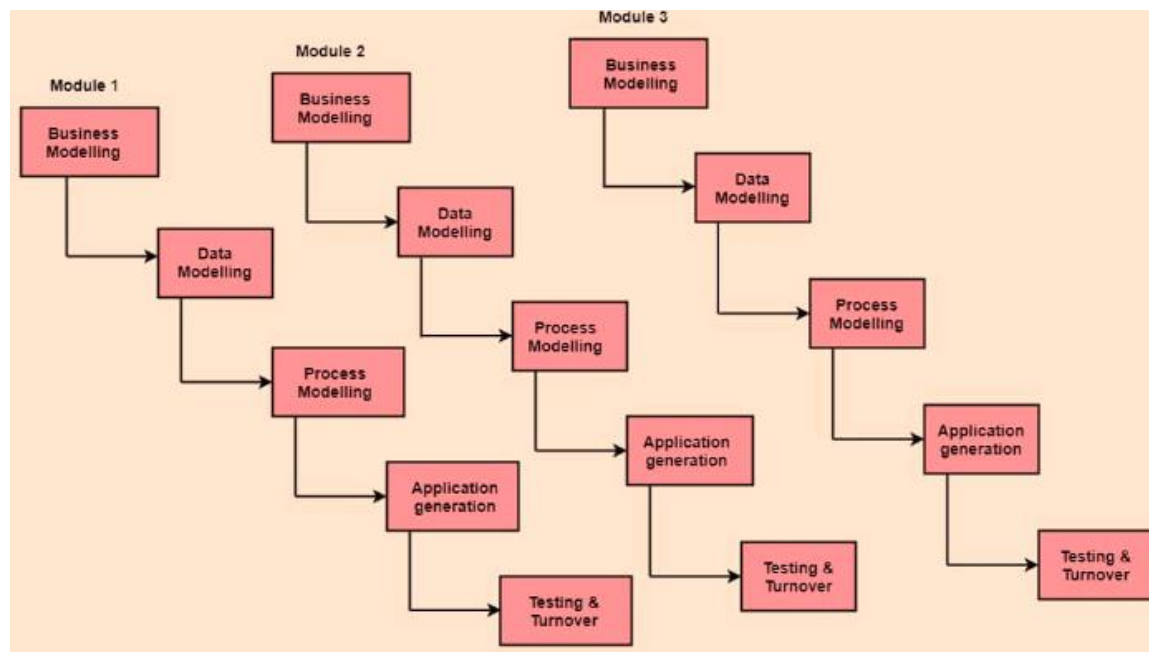
- Delivery is available in short duration (within @ 60- 90 days)
- Multiple teams work in parallel in activities such as modeling , construction of different modules (sub systems)
- When the requirements are fully understood and the component-based construction approach is adopted then the RAD model is used.

RAD model

- A software project can be implemented using this model if the project can be broken down into small modules wherein each module can be assigned independently to separate teams.
- These modules can finally be combined to form the final product.
- May be inconsistent
- Integration could be difficult

The RAD Model (Rapid Application Development)





RAD

- Sometimes referred to as the rapid prototyping model.

Features:

- Decrease the time taken and the cost incurred to develop software systems.
- Facilitate accommodating change requests as early as possible.
 - Before large investments have been made in development and testing
- A way to reduce development time and cost, and yet have flexibility to incorporate changes:
- Make only short term plans and make heavy reuse of existing code.

For which Applications is RAD Suitable?

- Customized product developed for one or two customers only
- Performance and reliability are not critical.
- The system can be split into several independent modules.

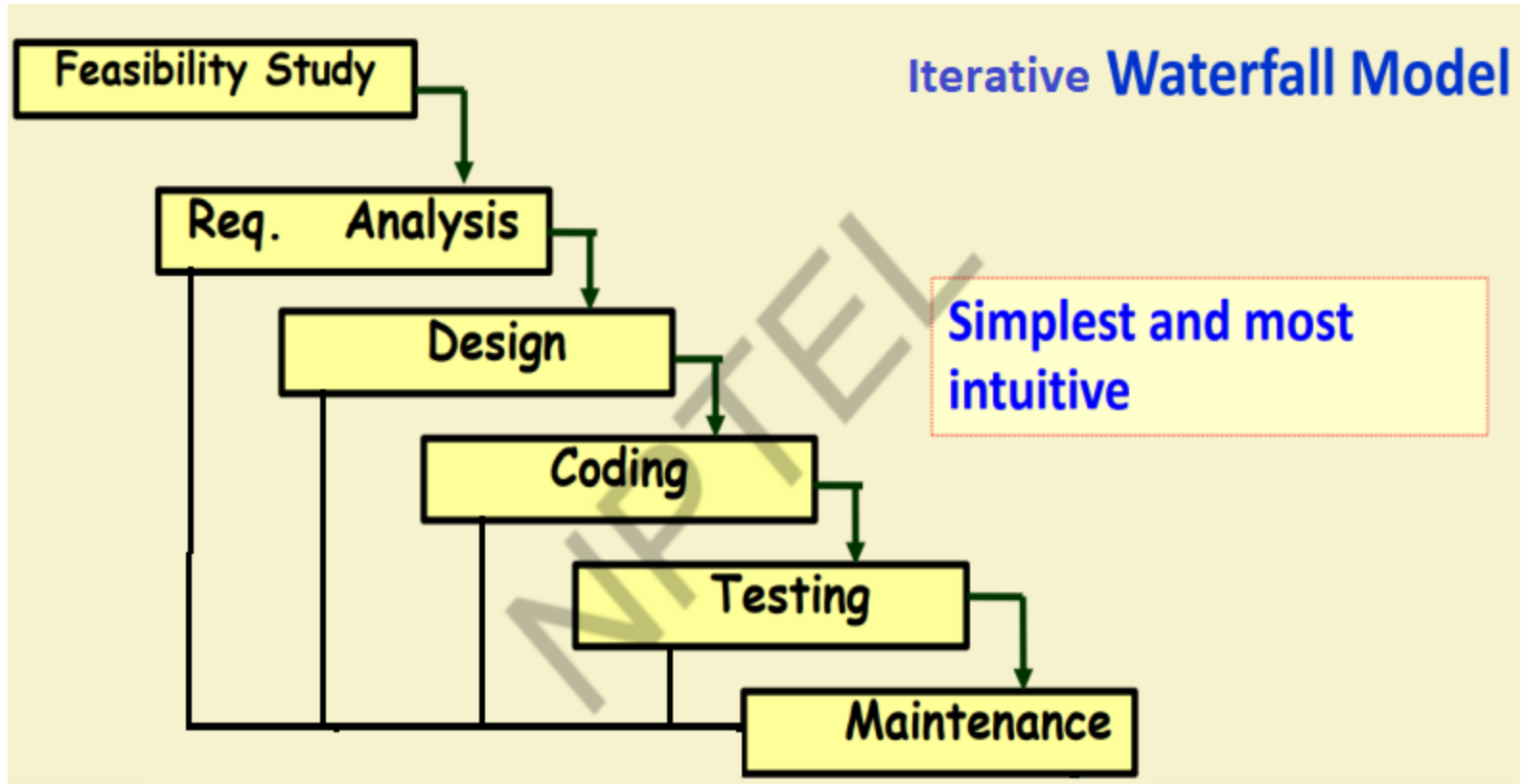
Iterative Model

**Software development stages are
repeated to IMPROVE
performance**

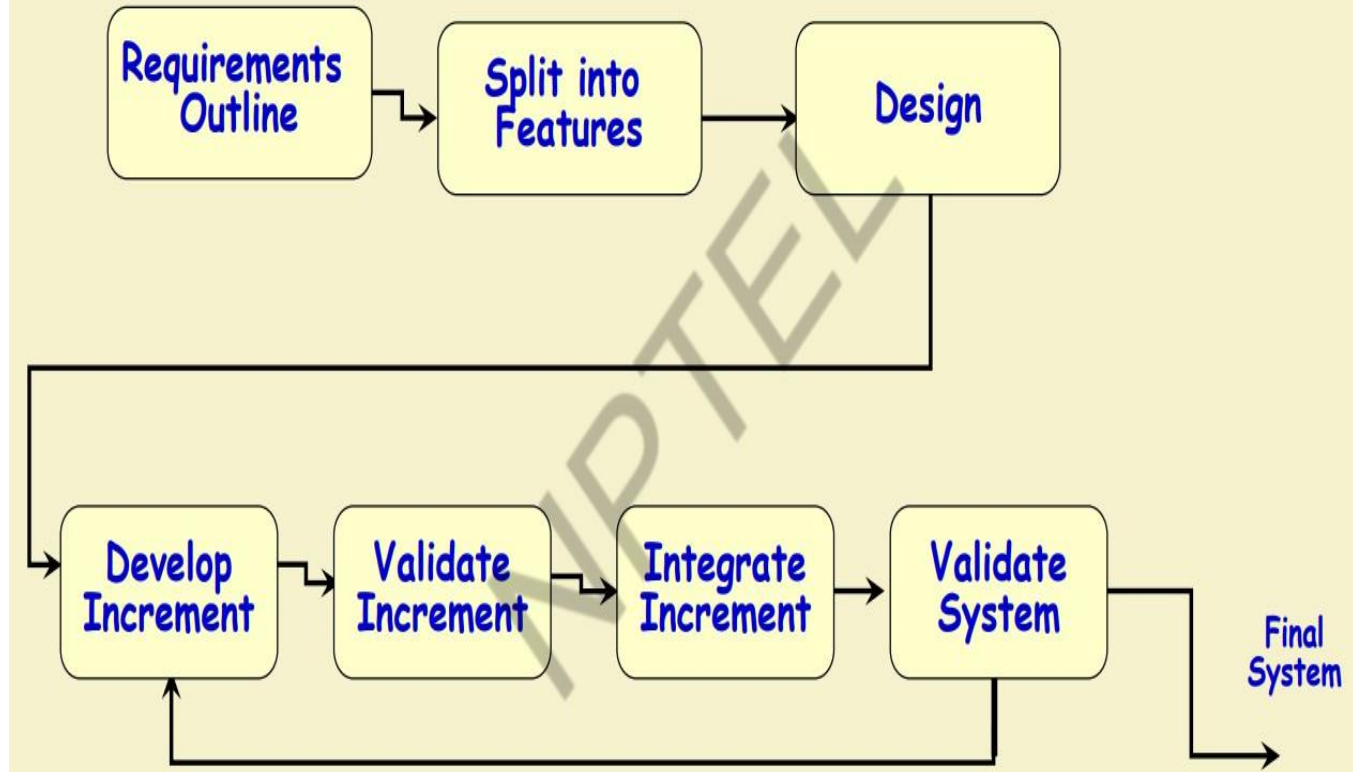
Fix bugs (problems)

**Focus is NOT on adding functionalities
(as against of Incremental model)**

Example



Incremental Model

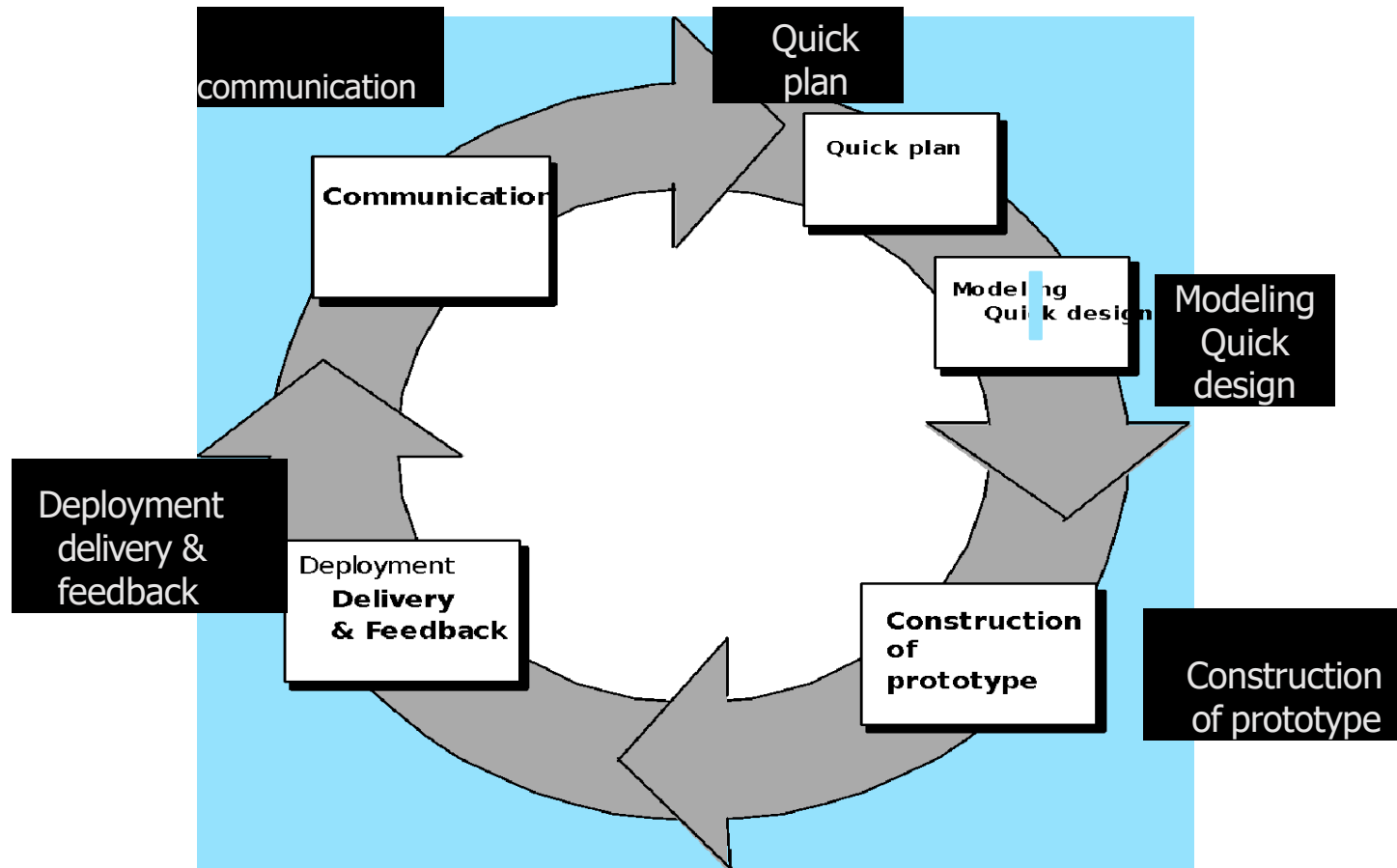


Incremental model

- Waterfall: single release
 - Iterative: many releases
-
- First increment: core functionality
 - Successive increments: add/fix functionality
 - Final increment: the complete product

Each iteration: a short mini-project with a separate lifecycle – e.g., waterfall

Evolutionary Models: Prototyping



These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

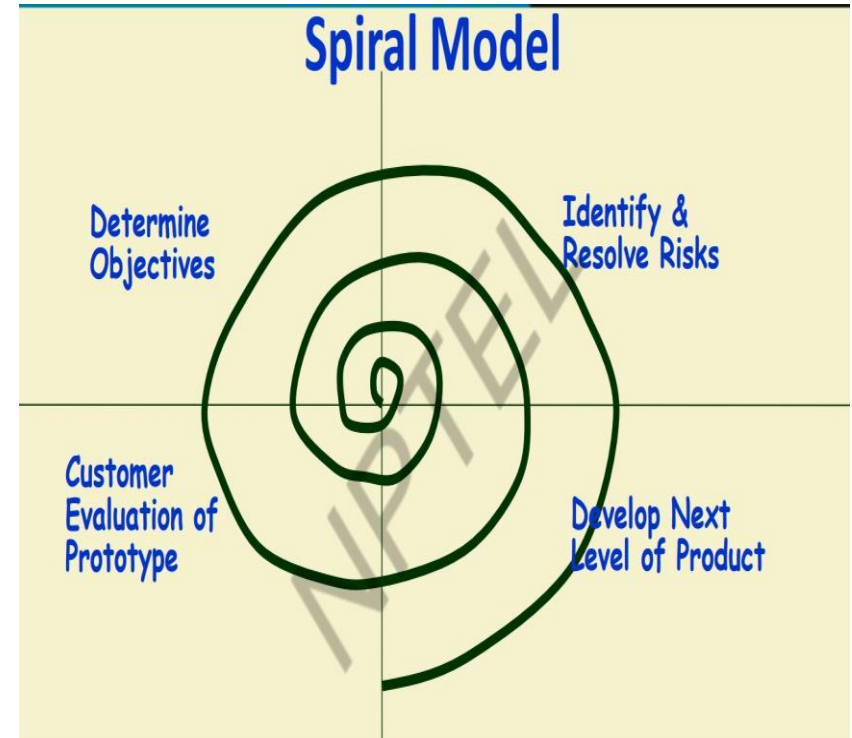
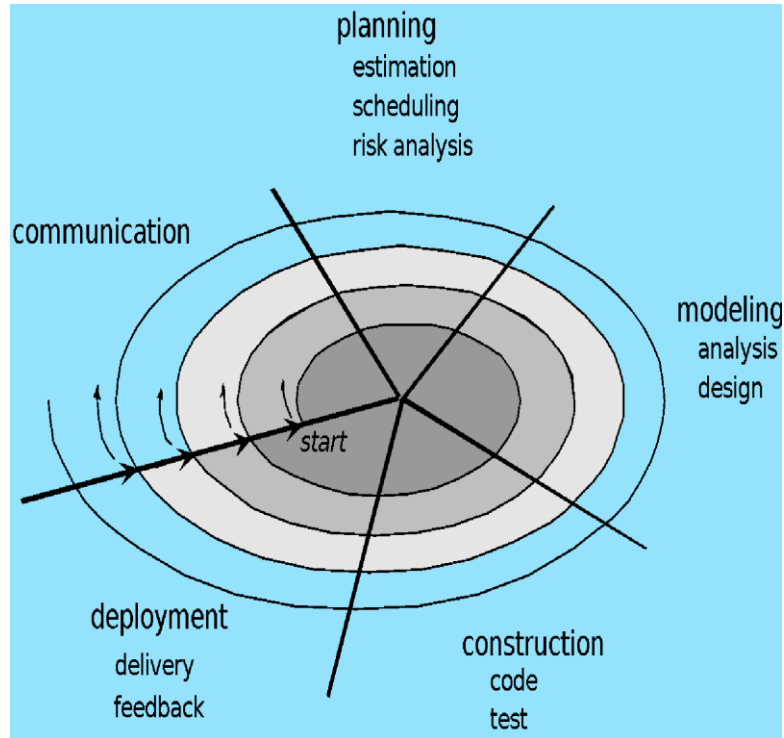
Evolutionary Prototype Model

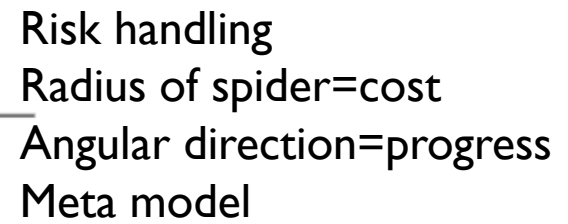
- It is a combination of iterative and incremental model.
 - Could be used for Understanding
 - Requirements
 - Functioning
 - Testing new technology
- The code/ product MAY NOT be used in the final product

Spiral model

- Originally proposed by Barry Boehm, the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled.
- It is a risk-driven process model generator that is used to guide multi-stakeholder concurrent engineering of software intensive systems.
- The exact number of loops of the spiral is unknown and can vary from project to project.
- Each loop of the spiral is called a **phase** of the software development process.

Boehm's Spiral model





Boehm's Spiral Model

- Documents and product are available after each iteration
- Improves & adds functionalities after each cycle
- Requires ALL types of human resources at regular frequency



Advantages

- Risk handling
- Flexible
- Large projects
- Customer satisfaction

Disadvantages:

- Complex
- Expensive
- Too much risk analysis

Agile Model

- What is Agile Software Development?
- **Agile:** Easily/Quickly moved, light, nimble (smart), active software processes
- How agility achieved?
 - Fitting the process to the project
 - Avoidance of things that waste time
- To overcome the shortcomings of the waterfall model of development.
 - Proposed in mid-1990s

Agile Model

- The agile model was primarily designed:
 - To help projects to adapt to change requests
- In the agile model:
 - The requirements are decomposed into many small incremental parts that can be developed over one to four weeks each.

Agile

- Effective (rapid and adaptive) **response to change** (team members, new technology, requirements)
- Effective **communication** in structure and attitudes among all team members, technological and business people, software engineers and managers
- Involving **the customer into the team**, Working as a **team**
- Planning in an uncertain world has its limits and plan must be **flexible**
- Organizing a **team** so that it **is in control of the work performed**
- Emphasize an **incremental delivery** strategy as opposed to intermediate products that gets working software to the customer as rapidly as feasible.
- **Rapid**, incremental delivery of software
- The development guidelines stress **delivery over analysis and design** although these activities are not discouraged, and **active and continuous communication** between developers and customers

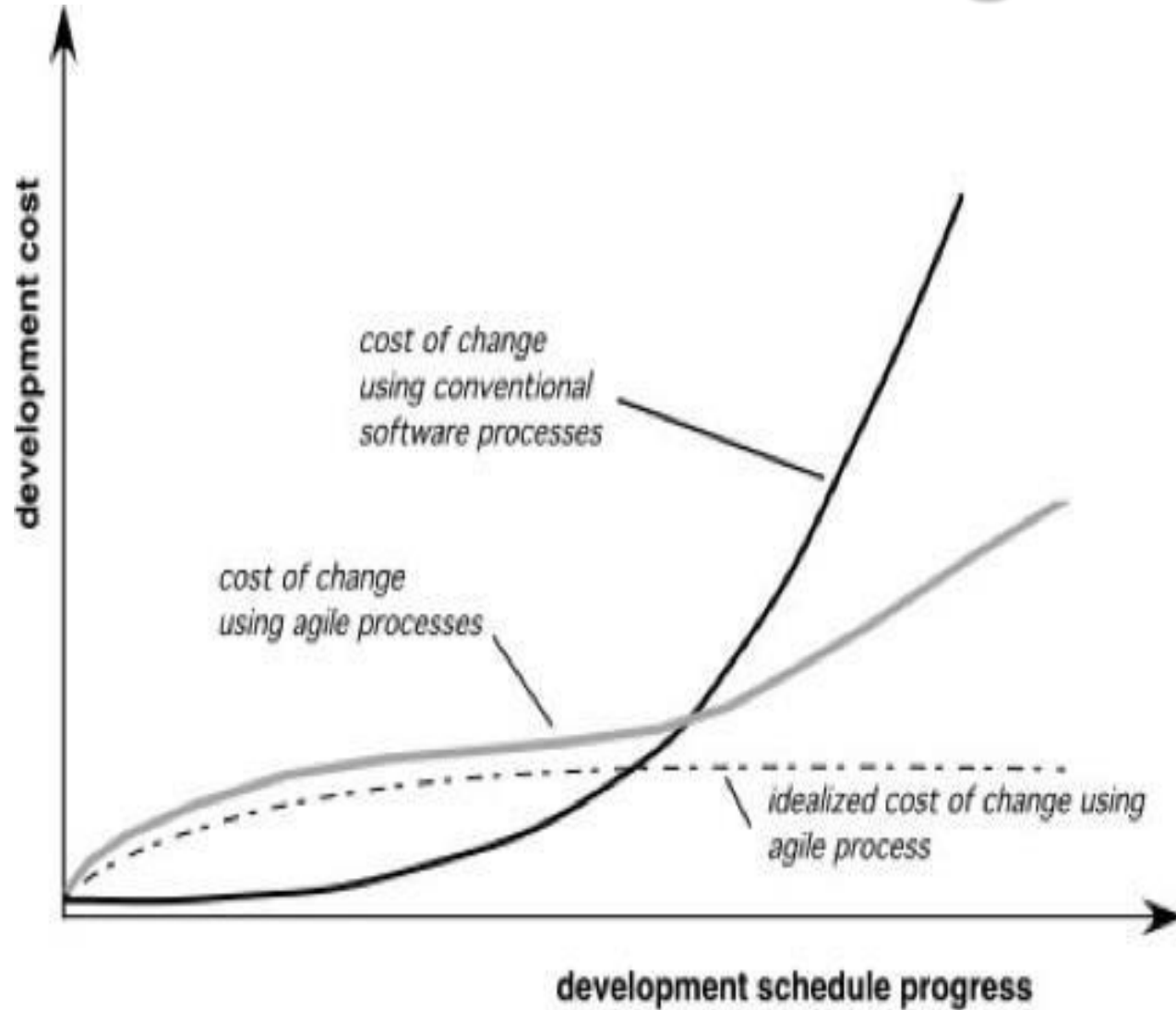
Benefits of Agility

Relatively easy to accommodate a change when a team is gathering requirements early in a project.

The costs of doing this work are minimal.

A well-designed agile process may “flatten” the cost of change curve by coupling incremental delivery with agile practices such as continuous unit testing and pair programming.

Cost of Change in Agile



Agile Process

- A **customer descriptions** of what is required (scenarios). Agile processes is characterized with some assumptions:
- Recognizes that plans are short-lived
 - some requirements will persist
 - some will change
 - Customer priorities will change
- Develops software iteratively with a heavy **emphasis on construction** activities
 - Design and construction are interleaved, hard to say how much design is necessary before construction
 - Design models are proven as they are created
- Analysis, design, construction and testing are not predictable
 - Adapt as changes occur due to unpredictability
- Delivers multiple 'software increments', deliver an operational prototype or portion of an OS to collect

Principles of Agility

1. Highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Allow changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale
4. Business people and developers must work together daily throughout the project . The most efficient and effective method of conveying information to and within a development team is face-to-face conversation
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

Principles of Agility

7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development, as the sponsors, developers, and users should work together.
9. Continuous attention to technical excellence and good design enhances agility
10. Simplicity – the art of maximizing results & less hard work (by removing unnecessary tasks)
11. The best architectures, requirements, and designs emerge from self-organizing teams
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly

Human factors in Agile Development Process

Development would be people focused than process

- **Competence** (talent, skills, knowledge)
- **Common focus** (deliver a working software increment)
- **Collaboration** (peers and stakeholders)
- **Decision-making ability** (freedom to control its own destiny)
- **Fuzzy problem-solving ability** (ambiguity and constant changes, today problem may not be tomorrow's problem)
- **Mutual trust** and respect
- **Self-organization** (themselves for the work done, process for its local environment, the work schedule)

Agile Methodologies

- XP (Extreme Programming)
- Scrum
- Crystal
- DSDM
- ASD
- FDD

Extreme Programming (Agile Approach)

- The most widely used agile process
 - **XP Planning**
 - **XP Design**
 - **XP Coding**
 - **XP Testing**

Extreme Programming (XP)

- The most widely used agile process
- **XP Planning**
 - Begins with the listening, leads to creation of “user stories” that describes required output, features, and functionality. Customer assigns a **value**(i.e., a priority) to each story.
 - Agile team assesses each story and assigns a **cost** (development weeks. If more than 3 weeks, customer asked to split into smaller stories)
 - Working together, stories are grouped for a deliverable increment next release.
- A commitment (stories to be included, delivery date and other project matters) is made. Three ways:
 1. Either all stories will be implemented in a few weeks,
 2. High priority stories first, or
 3. the riskiest stories will be implemented first.

- After the first increment “**project velocity**” (number of stories implemented during the first release) is used to help define subsequent delivery dates for other increments.
- Customers can add stories, delete existing stories, change values of an existing story, split stories as development work proceeds.

Extreme Programming (XP)

- **XP Design** (occurs both before and after coding as refactoring is encouraged)!
 - Follows the Keep It Simple(KIS) principle)Nothing more nothing less than the story.
 - Encourage the use of **CRC (class-responsibility-collaborator)** cards in an object-oriented context.
 - The only design work product of XP. They identify and organize the classes that are relevant to the current software increment
 - For difficult design problems, suggests the creation of **“spike solutions”**—a design prototype for that portion is implemented and evaluated.
 - Encourages **“refactoring”**—an iterative refinement of the internal program design.
 - Does not alter the external behavior yet improve the internal structure.
 - Minimize chances of bugs. More efficient, easy to read

- **XP Coding**

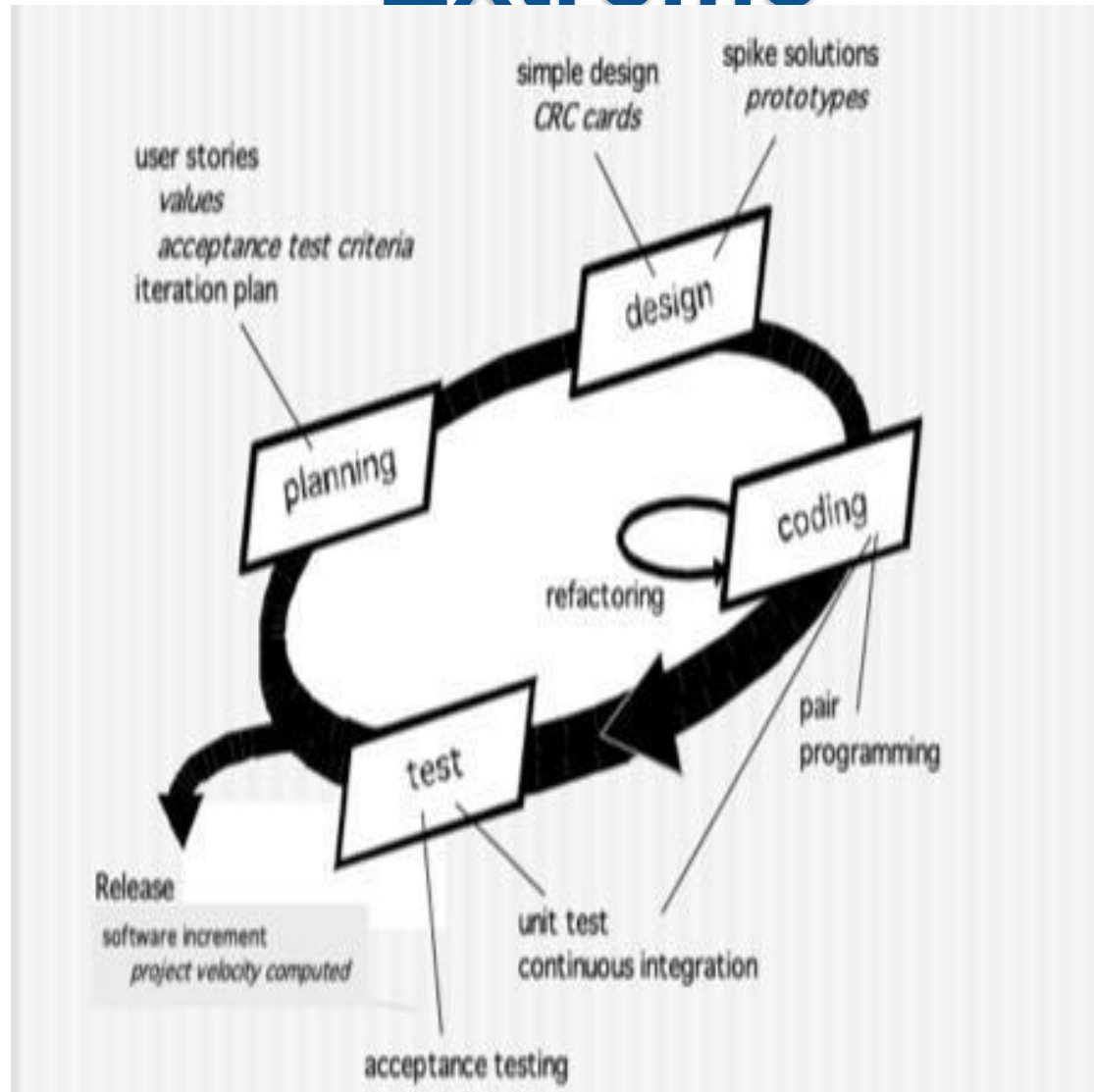
- Recommends the construction of a unit test for a story before coding commences. So implementer can focus on what must be implemented to pass the test
- Encourages “pair programming”. Two people work together at one workstation. Real time problem solving, real time review for quality assurance. Take slightly different roles
- Continuous integration

- **XP Testing**

- All unit tests are executed daily and ideally should be automated. Regression tests are conducted to test current and previous components
- “Acceptance tests” are defined by the customer and executed to assess customer visible functionality

Extreme

(XP)



Merits & Limitations of XP

- **Requirements volatility**: customer is an active member of XP team, changes to requirements are requested informally and frequently
- **Conflicting customer needs**: different customers' needs need to be assimilated. Different vision or beyond their authority
- **Requirements are expressed informally**: Use stories and acceptance tests are the only explicit manifestation of requirements. Formal models may avoid inconsistencies and errors before the system is built. Proponents said changing nature makes such models obsolete as soon as they are developed
- **Lack of formal design**: XP deemphasizes the need for architectural design. Complex systems need overall structure to exhibit quality and maintainability. Proponents said incremental nature limits complexity as simplicity is a core

Crystal

A resource limited, cooperative game of invention and communication, with a primary goal of delivering useful, working software and a secondary goal of setting up for the next game.

A set of methodologies, each with core elements that are common to all, and roles, process patterns, work products, and practice that are unique to each

Distinguishing features:

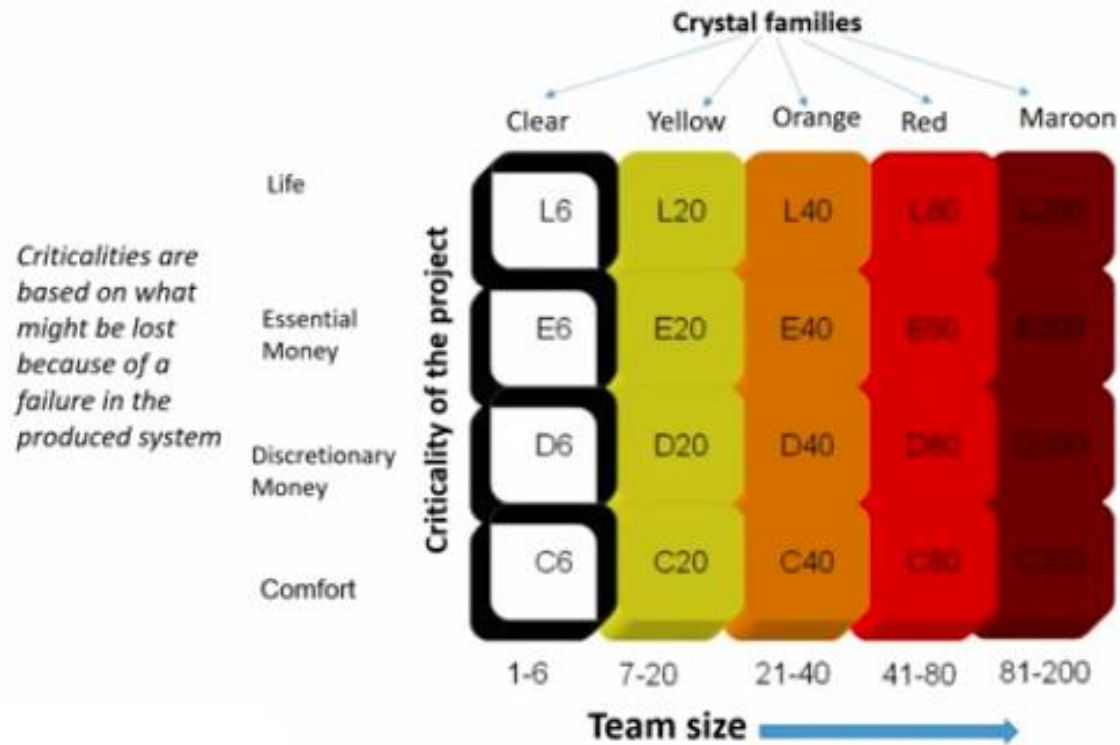
- A collection of process models that allow “maneuverability” based on problem characteristics.
- Face-to-face communication is emphasized.
- Suggests the use of “reflection workshops” to review the work habits of the team.

Crystal Model

- It is a family of methodologies
- Primary focus on people working on project
- Suitable approach for your project depends on:
 - Team size
 - Criticality
 - Priority of the project



Each project may require a slightly tailored set of policies, practices, and processes in order to meet the project's unique characteristics.



Adaptive Software Development (ASD)

Focusing on-

human collaboration and team self-organization as a technique to build complex software and system.

Distinguishing features

- **Mission-driven** planning
- **Component-based** focus
- Uses “**time-boxing**” focus of Project Management Concepts
- Explicit **consideration of risks**
- Emphasizes **collaboration** for requirements gathering
- Emphasizes “**learning**” throughout the process

Three Phases of ASD

1. Speculation: project is initiated and adaptive cycle planning is conducted.

- Adaptive cycle planning uses project initiation information- the customer's mission statement, project constraints (e.g. delivery date), and basic requirements to define the set of release cycles (increments) that will be required for the project.
- Based on the information obtained at the completion of the first cycle, the plan is reviewed and adjusted so that planned work better fits the reality.

Three Phases of ASD

2. Collaborations are used to multiply their talent and creative output beyond absolute number ($1+1>2$).

It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking

It is a matter of trust.

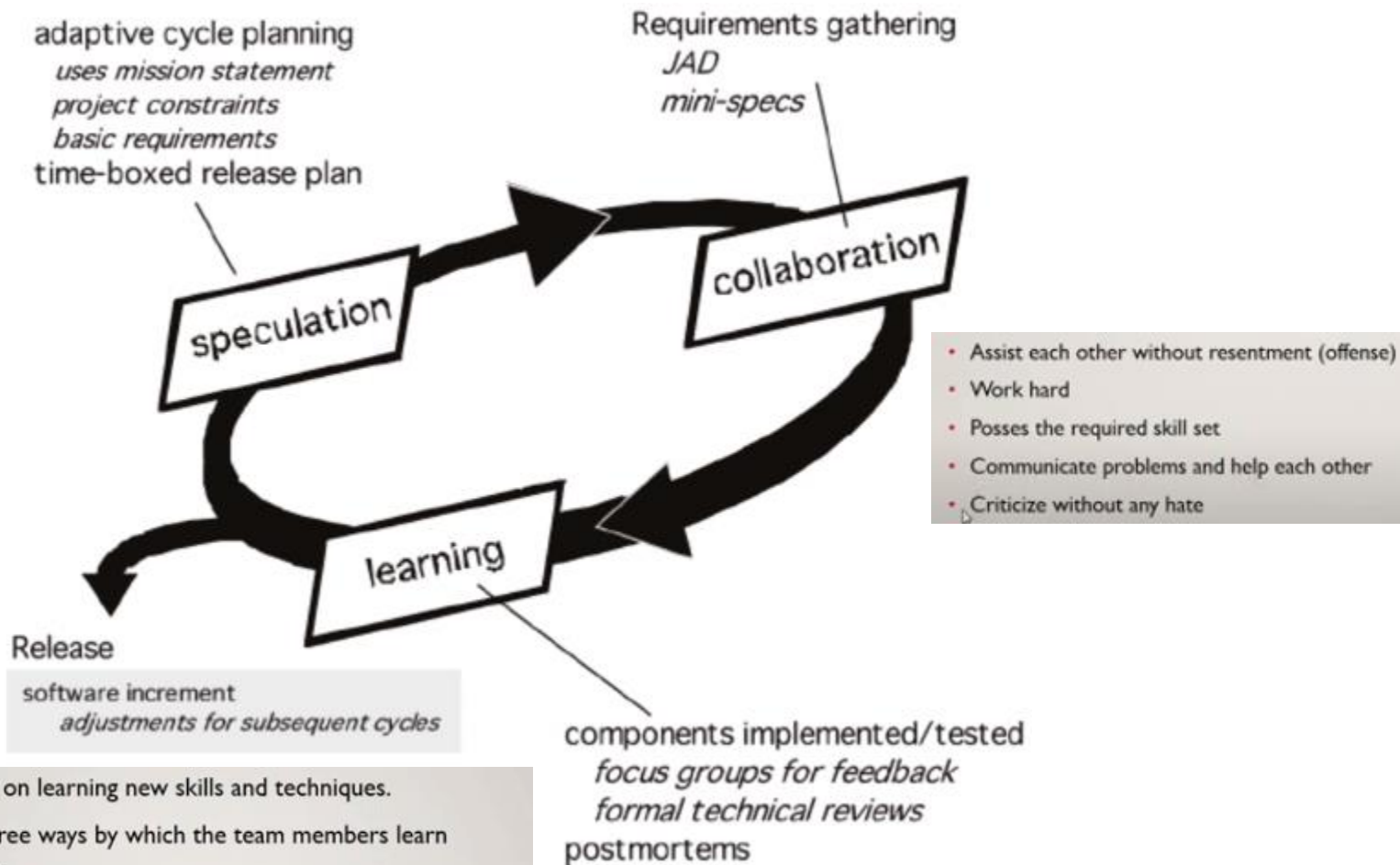
- criticize without animosity
- assist without resentments
- work as hard as or harder than they do
- have the skill set to contribute to the work at hand
- communicate problems or concerns in a way that leads to effective action.

Three Phases of ASD

3. Learning: As members of ASD team begin to develop the components, the emphasis is on “learning”.

- software developers often overestimate their own understanding of the technology, the process, and the project and that learning will help them to improve their level of real understanding.
- Three ways: focus groups, technical reviews and project post-mortems

Three Phases of ASD



Dynamic Systems Development Method (DSDM)

Provides a framework for building and maintaining systems which meet **tight time constraints** through the use of incremental prototyping in a controlled project environment

Works in Pareto principle-20% efforts =80 % result =100%

80% application can deliver in 20% of time.

- DSDM—distinguishing features

- Similar in most respects to XP and/or ASD

- **Guiding principles**

1. Active user involvement is imperative.

2. DSDM teams must be empowered to make decisions

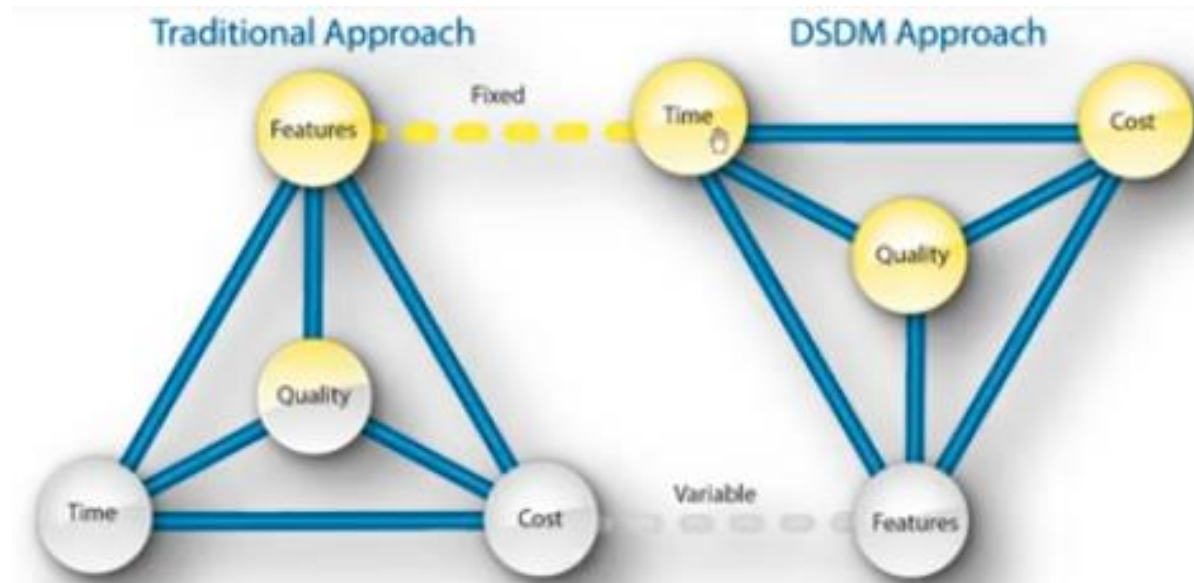
3. The focus is on frequent delivery of products

4. Fitness for business purpose is the essential criterion for acceptance of deliverables

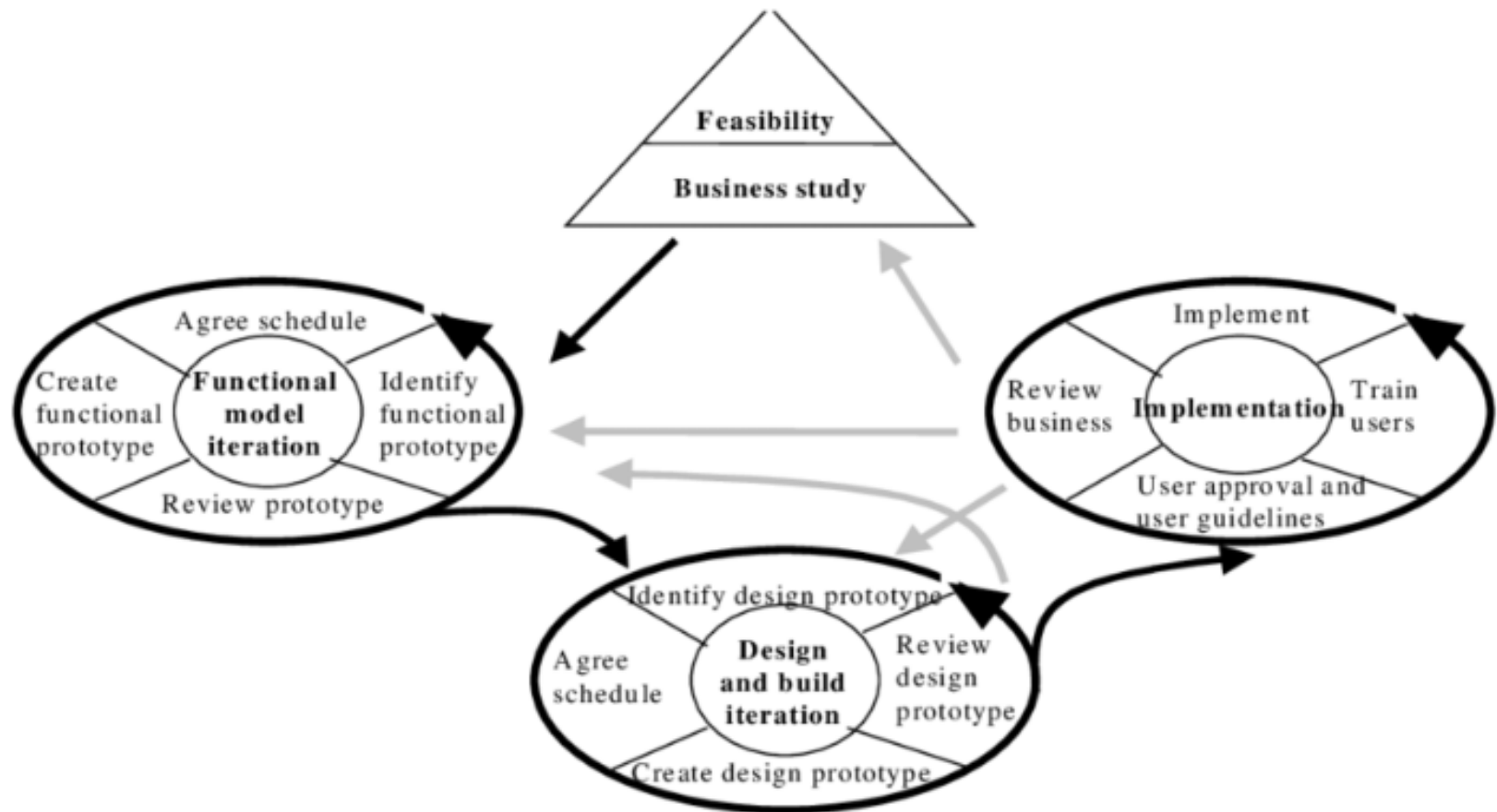
5. Iterative and incremental development is necessary to converge on an accurate business solution

6. All changes during development are reversible

Comparison



DSDM Life Cycle



Scrum

Features

- Development work is partitioned into **“packets”**
- **Testing and documentation are on-going** as the product is constructed
- Work units occurs in **“sprints”** and is derived from a **“backlog”** of existing changing prioritized requirements
- Changes are not introduced in sprints (short term but stable) but in backlog
- **Meetings are very short** (15 minutes daily) and sometimes conducted without chairs (what did you do since last meeting? What obstacles are you encountering? What do you plan to accomplish by next meeting?)
- **“demos”** are delivered to the customer with the time-box allocated. May not contain all functionalities. So

Scrum model

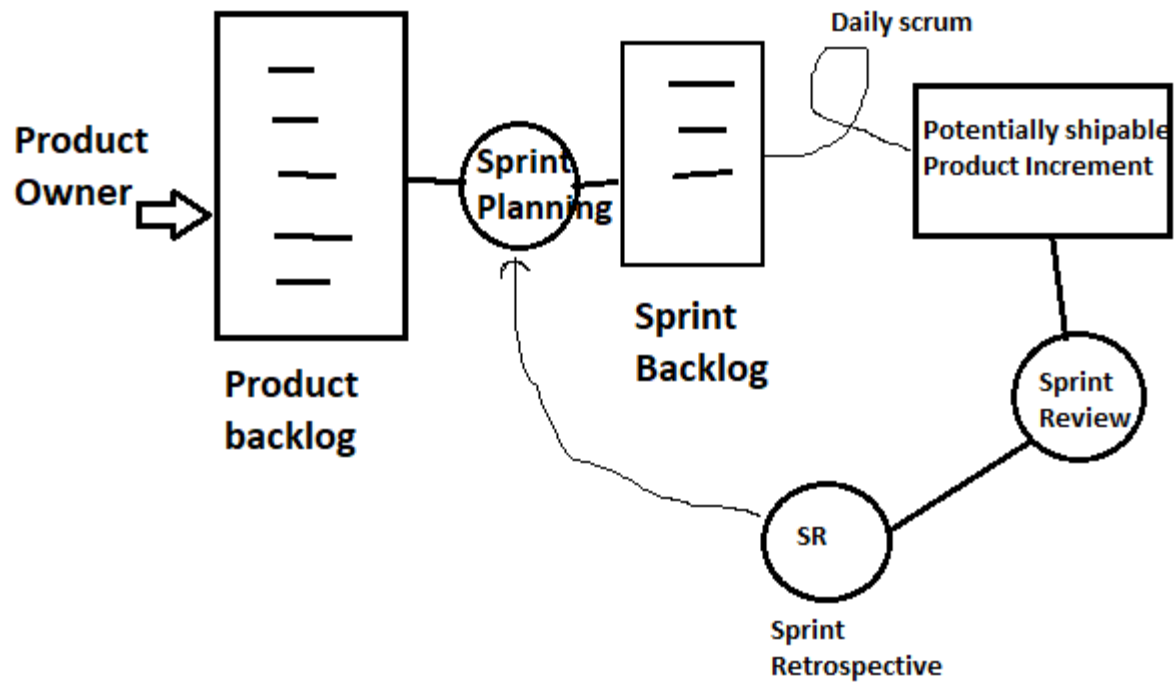
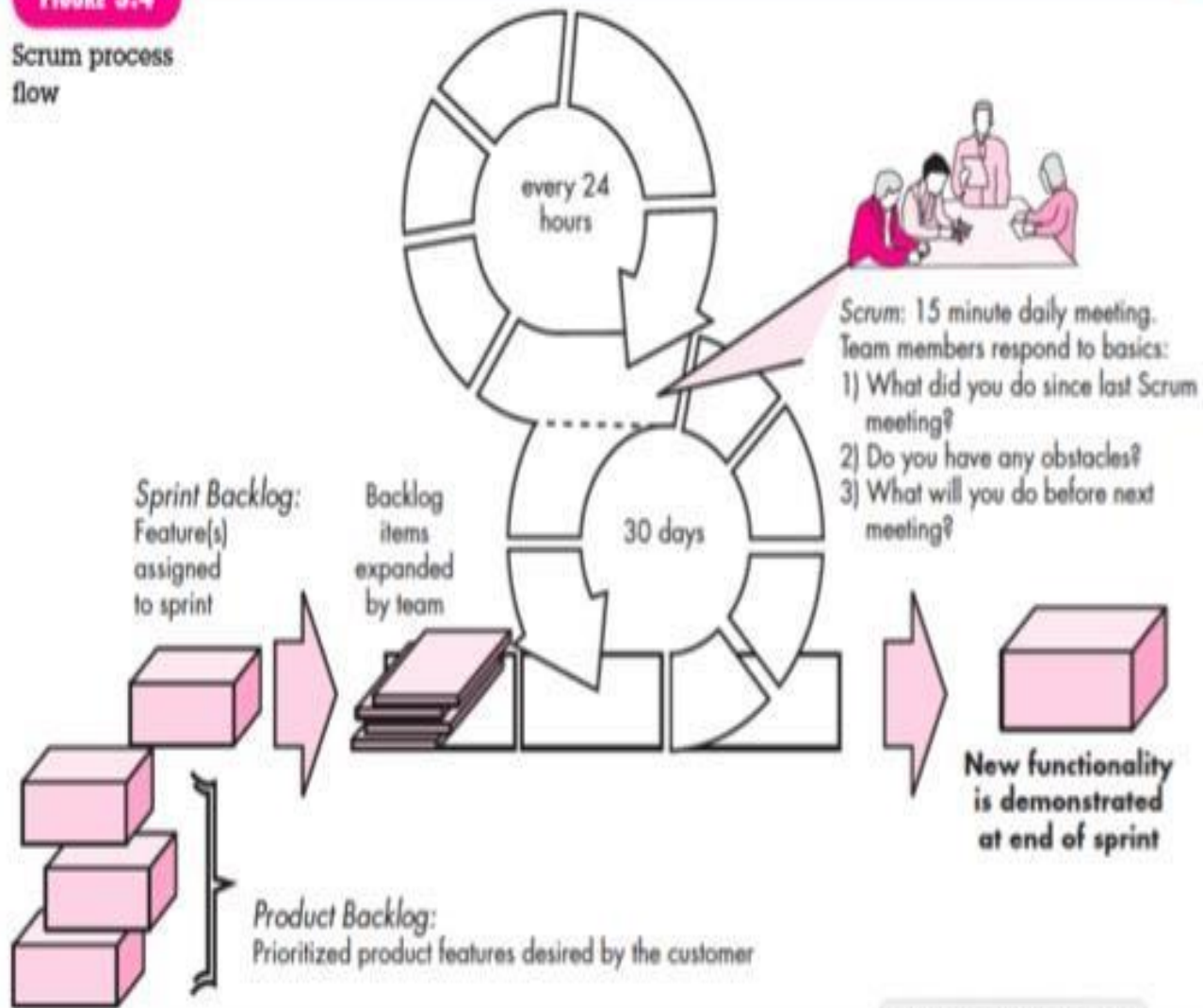


FIGURE 3.4

Scrum process flow



Assignment

- Compare various Software Development Life Cycle models
- Linear, Iterative, Spiral, Prototype, Agile

Feature Driven Development (FDD)

Features:

Emphasis is on defining **“features”** which can be organized hierarchically

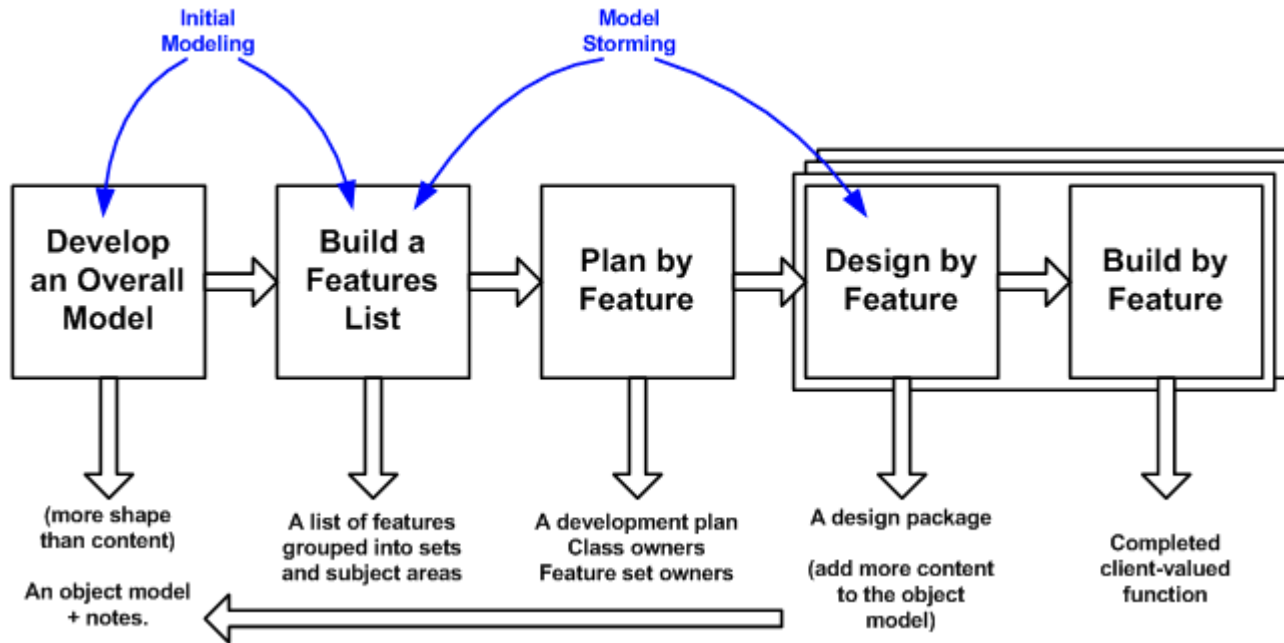
a **feature** “is a client-valued function that can be implemented in two weeks or less.”

Uses a **feature template**

- **<action> the <result> <by | for | of | to> a(n) <object>!**
- E.g. Add the product to shopping cart.
- Display the technical-specifications of the product.
- Store the shipping-information for the customer.

A **features list** is created and **“plan by feature”** is conducted Design and construction merge in FDD

FDD



Example: Library Management System

Features:

- Add a new book to the catalogue
- Update book information
- Register a new member
- Record a book loan
- Process a book return
- Generate overdue notices

References

- **Chapter 1:** Roger Pressman, “Software Engineering: A Practitioner’s Approach”, McGraw Hill, ISBN 0-07- 337597-7

